

Contents

1. Contest	1
2. Data Structures	2
3. Graph	11
4. Geometry	22
5. Number Theory	27
6. Strings	30
7. Theory	31

1. Contest

1.1. rng.h

mt19937_64

```
rng(chrono::steady_clock::now().time_since_epoch().count());
```

```
int random(int a, int b) {
    if (a > b)
        return 0;
    return a + rng() % (b - a + 1);
}
double random_double(double a, double b) {
    return a + (b - a) * (rng() / (double)rng.max());
}
```

1.2. template.h

```
#include <bits/stdc++.h>
```

```
#define tc \
    int t; \
    cin >> t; \
    while (t--) \
        solve();
using namespace std;
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
```

```
using namespace __gnu_pbds;
template <class T, class U = null_type, class chash = hash<T>>
using hset = gp_hash_table<T, U, chash>;
template <class T, class U = null_type, class cmp = less<T>>
using oset = tree<T, U, cmp, rb_tree_tag,
tree_order_statistics_node_update>;
#define MOD 1000000007
#define int long long
#define pa pair<int, int>
#define fr(i, a, b) for (int i = (a); i < (b); i++)
#define fnr(i, a, b) for (int i = (a); i >= (b); i--)
#define vi vector<int>
#define vpi vector<pa>
#define vvi vector<vi>
#define pb push_back
#define all(s) s.begin(), s.end()
#define set_bits \
    __builtin_popcountll // tells the number of
setbits or number of 1s
#define zero_bef __builtin_clzll // number of leading zeroes
#define sz(a) (int)a.size()
#define print(v) \
    for (auto printi : v) \
        cout << printi << ' '
// make a custom template
```

```
using T = long long; // Generic type for templates
using ldbl = long double;
```

```
void solve() {}
```

```
signed main() {
    ios::sync_with_stdio(false);
    cin.tie(NULL);
```

```
    return 0;
}
```

2. Data Structures

2.1. FenwickTree.h

```
template <typename T>
class FenwickTree {
public:
    int n;
    vector<T> bit;
    // 1 based fenwick tree, bit[1] = a[1]
    FenwickTree(int size) : n(size), bit(n + 1, T{}) {}

    FenwickTree(const vector<T>& arr) :
    FenwickTree((int)arr.size() - 1) {
        build(arr);
    }

    void build(const vector<T>& arr) {
        bit.assign(n + 1, T{});
        for (int i = 1; i <= n; ++i) {
            bit[i] += arr[i];
            int parent = i + (i & -i);
            if (parent <= n) bit[parent] += bit[i];
        }
    }

    void update(int idx, T delta) {
        for (; idx <= n; idx += idx & -idx)
            bit[idx] += delta;
    }

    T query(int idx) const {
        T result{};
        for (; idx > 0; idx -= idx & -idx)
```

```
        result += bit[idx];
    return result;
}
```

```
T query(int left, int right) const {
    return query(right) - query(left - 1);
}
};
```

```
template <typename T>
class FenwickTree2D {
public:
    int n, m;
    vector<vector<T>> bit;

    // 1 based 2D Fenwick tree. bit[i][j] covers the rectangle
    // ending at
    // (i, j) with dimensions (i & -i) by (j & -j).
    FenwickTree2D(int rows, int columns)
        : n(rows), m(columns), bit(n + 1, vector<T>(m + 1, T{}))
    {}

    FenwickTree2D(const vector<vector<T>>& arr)
        : FenwickTree2D((int)arr.size() - 1, (int)arr[0].size() -
1) {
        build(arr);
    }

    // how to use ?
    // write in code
    // FenwickTree2D<int>ft(#rows, #columns);
    // ft.build(arr);
    void build(const vector<vector<T>>& arr) {
        bit.assign(n + 1, vector<T>(m + 1, T{}));
        for (int i = 1; i <= n; ++i)
            for (int j = 1; j <= m; ++j)
                update(i, j, arr[i][j]);
```

```

    }

    void update(int row, int column, T delta) {
        for (int i = row; i <= n; i += i & -i)
            for (int j = column; j <= m; j += j & -j)
                bit[i][j] += delta;
    }

    T prefix_sum(int row, int column) const {
        T result{};
        for (int i = row; i > 0; i -= i & -i)
            for (int j = column; j > 0; j -= j & -j)
                result += bit[i][j];
        return result;
    }

    T rectangle_sum(int top, int left, int bottom, int right)
    const {
        return prefix_sum(bottom, right) - prefix_sum(top - 1,
right) -
            prefix_sum(bottom, left - 1) + prefix_sum(top - 1,
left - 1);
    }
};

template <typename T>
class RangeFenwickTree {
public:
    int n;
    vector<T> bit1, bit2;

    RangeFenwickTree(int size) : n(size), bit1(n + 1, T{}),
bit2(n + 1, T{}) {}

    RangeFenwickTree(const vector<T>& arr) :
RangeFenwickTree((int)arr.size() - 1) {

```

```

        build(arr);
    }

    void build(const vector<T>& arr) {
        bit1.assign(n + 1, T{});
        bit2.assign(n + 1, T{});
        for (int i = 1; i <= n; ++i)
            range_add(i, i, arr[i]);
    }

    void range_add(int left, int right, T value) {
        add(bit1, left, value);
        add(bit1, right + 1, -value);
        add(bit2, left, value * (left - 1));
        add(bit2, right + 1, -value * right);
    }

    T prefix_sum(int idx) const {
        return value(bit1, idx) * idx - value(bit2, idx);
    }

    T range_sum(int left, int right) const {
        return prefix_sum(right) - prefix_sum(left - 1);
    }

private:
    void add(vector<T>& bit, int idx, T value) {
        for (; idx <= n; idx += idx & -idx)
            bit[idx] += value;
    }

    T value(const vector<T>& bit, int idx) const {
        T result{};
        for (; idx > 0; idx -= idx & -idx)
            result += bit[idx];
        return result;
    }

```

```

    }
};

```

2.2. LazySegTree.h

```

class LazySegTree {
public:
    int n;
    vector<T> tree, lazy;
    // ---- in cases of assignment question jaha pe value change
    krni like assign
    // val p from [l,r] vector<bool> marked; // to track pending
    assignment

    LazySegTree(int size) : n(size) {
        tree.resize(4 * n + 1);
        lazy.resize(4 * n + 1, 0);
        // --- Uncomment when u need that assign val p in l,r vale
        // marked.resize(4 * n + 1, false);
    }

    void build(int node, int start, int end, const vector<T>&
arr) {
        if (start == end) {
            tree[node] = arr[start];
        } else {
            int mid = (start + end) / 2;
            build(2 * node + 1, start, mid, arr);
            build(2 * node + 2, mid + 1, end, arr);
            tree[node] = merge(tree[2 * node + 1], tree[2 * node +
2]);
        }
    }

    void apply_lazy(int node, int start, int end) {
        if (lazy[node] != 0) {
            tree[node] += (end - start + 1) * lazy[node];

```

```

        if (start != end) {
            lazy[2 * node + 1] += lazy[node];
            lazy[2 * node + 2] += lazy[node];
        }
        lazy[node] = 0;
    }
}

void update(int node, int start, int end, int l, int r, T
value) {
    apply_lazy(node, start, end);
    if (start > end || start > r || end < l)
        return;

    if (start >= l && end <= r) {
        tree[node] += (end - start + 1) * value;
        if (start != end) {
            lazy[2 * node + 1] += value;
            lazy[2 * node + 2] += value;
        }
        return;
    }

    int mid = (start + end) / 2;
    update(2 * node + 1, start, mid, l, r, value);
    update(2 * node + 2, mid + 1, end, l, r, value);
    tree[node] = merge(tree[2 * node + 1], tree[2 * node + 2]);
}

T query(int node, int start, int end, int l, int r) {
    apply_lazy(node, start, end);
    if (start > end || start > r || end < l)
        return identity();
    if (start >= l && end <= r)
        return tree[node];

```

```

    int mid = (start + end) / 2;
    return merge(query(2 * node + 1, start, mid, l, r),
                 query(2 * node + 2, mid + 1, end, l, r));
}

//
=====
// ===== OPTIONAL MODE: RANGE ASSIGN
=====
// ===== (assignment vale question jusme purana add
nhi krna )
// =====
//
=====

/*
void apply_lazy(int node, int start, int end) {
    if (marked[node]) {
        tree[node] = (end - start + 1) * lazy[node];
        if (start != end) {
            lazy[2 * node + 1] = lazy[node];
            lazy[2 * node + 2] = lazy[node];
            marked[2 * node + 1] = marked[2 * node + 2] =
true;
        }
        marked[node] = false;
    }
}

void update(int node, int start, int end, int l, int r, T
value) {
    apply_lazy(node, start, end);
    if (start > end || start > r || end < l) return;

    if (start >= l && end <= r) {
        tree[node] = (end - start + 1) * value;

```

```

        if (start != end) {
            lazy[2 * node + 1] = lazy[2 * node + 2] = value;
            marked[2 * node + 1] = marked[2 * node + 2] =
true;
        }
        return;
    }

    int mid = (start + end) / 2;
    update(2 * node + 1, start, mid, l, r, value);
    update(2 * node + 2, mid + 1, end, l, r, value);
    tree[node] = merge(tree[2 * node + 1], tree[2 * node +
2]);
}
*/

private:
    T merge(T a, T b) { return a + b; } // can replace with min/
max/gcd
    T identity() { return 0; } // replace as needed
};

```

2.3. LazySegTree2D.h

```

template <typename T>
class LazySegTree2D {
public:
    int n, m;
    vector<vector<T>> tree, lazy;
    // Uncomment these for assignment mode
    // vector<vector<bool>> marked;

    LazySegTree2D(int n_, int m_) : n(n_), m(m_) {
        tree.assign(4 * n, vector<T>(4 * m, 0));
        lazy.assign(4 * n, vector<T>(4 * m, 0));
        // marked.assign(4 * n, vector<bool>(4 * m, false));
    }
}

```

```
// ===== BUILD =====

// here note we store vx and vy which are node of the seg
tree
// lx , rx are the range in x direction
// ly , ry are the range in y direction
void build_y(int vx, int lx, int rx, int vy, int ly, int ry,
             const vector<vector<T>>& a) {
    if (ly == ry) {
        if (lx == rx)
            tree[vx][vy] = a[lx][ly];
        else
            tree[vx][vy] = merge(tree[vx * 2][vy], tree[vx * 2 + 1]
[vy]);
    } else {
        int my = (ly + ry) / 2;
        build_y(vx, lx, rx, vy * 2, ly, my, a);
        build_y(vx, lx, rx, vy * 2 + 1, my + 1, ry, a);
        tree[vx][vy] = merge(tree[vx][vy * 2], tree[vx][vy * 2 +
1]);
    }
}

void build_x(int vx, int lx, int rx, const vector<vector<T>>&
a) {
    if (lx != rx) {
        int mx = (lx + rx) / 2;
        build_x(vx * 2, lx, mx, a);
        build_x(vx * 2 + 1, mx + 1, rx, a);
    }
    build_y(vx, lx, rx, 1, 0, m - 1, a);
}

// ===== LAZY APPLY (ADD MODE)
=====
```

```
void apply_lazy_y(int vx, int vy, int ly, int ry) {
    if (lazy[vx][vy] != 0) {
        tree[vx][vy] += (ry - ly + 1) * lazy[vx][vy];
        if (ly != ry) {
            lazy[vx][vy * 2] += lazy[vx][vy];
            lazy[vx][vy * 2 + 1] += lazy[vx][vy];
        }
        lazy[vx][vy] = 0;
    }
}

// ===== UPDATE IN Y =====
void update_y(int vx, int lx, int rx,
              int vy, int ly, int ry,
              int y1, int y2, T val) {
    apply_lazy_y(vx, vy, ly, ry);
    if (ly > ry || ly > y2 || ry < y1) return;

    if (ly >= y1 && ry <= y2) {
        tree[vx][vy] += (ry - ly + 1) * val;
        if (ly != ry) {
            lazy[vx][vy * 2] += val;
            lazy[vx][vy * 2 + 1] += val;
        }
        return;
    }

    int my = (ly + ry) / 2;
    update_y(vx, lx, rx, vy * 2, ly, my, y1, y2, val);
    update_y(vx, lx, rx, vy * 2 + 1, my + 1, ry, y1, y2, val);
    tree[vx][vy] = merge(tree[vx][vy * 2], tree[vx][vy * 2 +
1]);
}

// ===== UPDATE IN X =====
void update_x(int vx, int lx, int rx,
```

```

        int x1, int x2, int y1, int y2, T val) {
    if (lx > rx || lx > x2 || rx < x1) return;

    if (lx >= x1 && rx <= x2) {
        update_y(vx, lx, rx, 1, 0, m - 1, y1, y2, val);
        return;
    }

    int mx = (lx + rx) / 2;
    update_x(vx * 2, lx, mx, x1, x2, y1, y2, val);
    update_x(vx * 2 + 1, mx + 1, rx, x1, x2, y1, y2, val);

    for (int vy = 1; vy < 4 * m; vy++) {
        tree[vx][vy] = merge(tree[vx * 2][vy], tree[vx * 2 + 1]
[vy]);
    }
}

// ===== QUERY IN Y =====
T query_y(int vx, int vy, int ly, int ry,
        int y1, int y2) {
    apply_lazy_y(vx, vy, ly, ry);
    if (ly > ry || ly > y2 || ry < y1) return identity();
    if (ly >= y1 && ry <= y2) return tree[vx][vy];

    int my = (ly + ry) / 2;
    return merge(
        query_y(vx, vy * 2, ly, my, y1, y2),
        query_y(vx, vy * 2 + 1, my + 1, ry, y1, y2));
}

// ===== QUERY IN X =====
T query_x(int vx, int lx, int rx,
        int x1, int x2, int y1, int y2) {
    if (lx > rx || lx > x2 || rx < x1) return identity();
    if (lx >= x1 && rx <= x2)

```

```

        return query_y(vx, 1, 0, m - 1, y1, y2);

    int mx = (lx + rx) / 2;
    T q1 = query_x(vx * 2, lx, mx, x1, x2, y1, y2);
    T q2 = query_x(vx * 2 + 1, mx + 1, rx, x1, x2, y1, y2);
    return merge(q1, q2);
}

private:
    T merge(T a, T b) { return a + b; } // can be min/max/gcd
    T identity() { return 0; } // depends on merge op
};

```

2.4. Mos.h

```

int BLOCK = DO_NOT_FORGET_TO_CHANGE_THIS;
struct Query {
    int l, r, id;
    Query(int _l, int _r, int _id) : l(_l), r(_r), id(_id) {}
    bool operator<(Query& o) {
        int mblock = l / BLOCK, oblock = o.l / BLOCK;
        return (mblock < oblock) or
            (mblock == oblock and mblock % 2 == 0 and r < o.r)
or
            (mblock == oblock and mblock % 2 == 1 and r > o.r);
    };
};

void solve() {
    vector<Query> queries;
    queries.reserve(q);
    for (int i = 0; i < q; i++) {
        int l, r;
        cin >> l >> r;
        l--, r--;
        queries.emplace_back(l, r, i);
    }
    sort(all(queries));
}

```

```

int ans = 0;
auto add = [&](int v) {};
auto rem = [&](int v) {};
vector<int> out(q); // Change out type if necessary
int cur_l = 0, cur_r = -1;
for (auto& [l, r, id] : queries) {
    while (cur_l > l)
        add(--cur_l);
    while (cur_l < l)
        rem(cur_l++);
    while (cur_r < r)
        add(++cur_r);
    while (cur_r > r)
        rem(cur_r--);
    out[id] = ans;
}
}

```

2.5. PersistentSegTree.h

```

struct Node {
    int val;
    Node *left, *right;
    Node(int v = 0, Node* l = nullptr, Node* r = nullptr)
        : val(v), left(l), right(r) {}
};

struct PersistentSegTree {
    int n;
    vector<Node*> version; // roots of each version

    PersistentSegTree(int n) : n(n) {
        version.push_back(build(0, n - 1)); // version[0]: initial
        (all 0)
    }

    Node* build(int l, int r) {

```

```

        if (l == r) return new Node(0);
        int m = (l + r) / 2;
        return new Node(0, build(l, m), build(m + 1, r));
    }

    // create new version with arr[idx] += val
    Node* update(Node* prev, int l, int r, int idx, int val) {
        if (l == r)
            return new Node(prev->val + val); // create new leaf
        node

        int m = (l + r) / 2;
        if (idx <= m)
            return new Node(prev->val + val,
                            update(prev->left, l, m, idx, val),
                            prev->right);
        else
            return new Node(prev->val + val,
                            prev->left,
                            update(prev->right, m + 1, r, idx, val));
    }

    int query(Node* node, int l, int r, int ql, int qr) {
        if (!node || ql > r || qr < l) return 0;
        if (ql <= l && r <= qr) return node->val;
        int m = (l + r) / 2;
        return query(node->left, l, m, ql, qr) +
               query(node->right, m + 1, r, ql, qr);
    }

    // user interface:
    void update_version(int prev_version, int idx, int val) {
        Node* new_root = update(version[prev_version], 0, n - 1,
                                idx, val);
        version.push_back(new_root);
    }
}

```

```

int query_version(int ver, int l, int r) {
    return query(version[ver], 0, n - 1, l, r);
}
};

```

2.6. SegTree.h

```

template <typename T>
// Iterative segment tree with inclusive range queries
class segtree {
public:
    int n;
    vector<T> tree;

    segtree(int size) : n(size), tree(2 * n, identity()) {}

    segtree(const vector<T>& arr) : n((int)arr.size()), tree(2 *
n) {
        build(arr);
    }

    void build(const vector<T>& arr) {
        n = (int)arr.size();
        tree.assign(2 * n, identity());
        copy(arr.begin(), arr.end(), tree.begin() + n);
        for (int node = n - 1; node > 0; --node)
            tree[node] = merge(tree[node << 1], tree[node << 1 | 1]);
    }

    void update(int idx, T value) {
        for (tree[idx += n] = value; idx > 1; idx >>= 1)
            tree[idx >> 1] = merge(tree[idx], tree[idx ^ 1]);
    }

    T query(int l, int r) {
        T left_result = identity(), right_result = identity();

```

```

        for (l += n, r += n + 1; l < r; l >>= 1, r >>= 1) {
            if (l & 1) left_result = merge(left_result, tree[l++]);
            if (r & 1) right_result = merge(tree[--r], right_result);
        }
        return merge(left_result, right_result);
    }
}

```

```

private:
    T merge(T a, T b) { return a + b; } // change to min/max/gcd
    T identity() { return 0; } // change to the merge
identity
};

```

2.7. SegTree2D.h

```

template <typename T>
class SegTree2D {
public:
    int n, m;
    vector<vector<T>> tree;
    vector<vector<T>> arr;

    SegTree2D(int _n, int _m) : n(_n), m(_m) {
        arr.assign(n, vector<T>(m, 0));
        tree.assign(4 * n, vector<T>(4 * m, 0));
    }

    void build_y(int node_x, int start_x, int end_x, int node_y,
int start_y, int end_y) {
        if (start_y == end_y) {
            if (start_x == end_x)
                tree[node_x][node_y] = arr[start_x][start_y];
            else
                tree[node_x][node_y] = merge(
                    tree[node_x * 2][node_y],
                    tree[node_x * 2 + 1][node_y]);
        } else {

```

```

    int mid_y = (start_y + end_y) / 2;
    build_y(node_x, start_x, end_x, node_y * 2, start_y,
mid_y);
    build_y(node_x, start_x, end_x, node_y * 2 + 1, mid_y +
1, end_y);
    tree[node_x][node_y] = merge(
        tree[node_x][node_y * 2],
        tree[node_x][node_y * 2 + 1]);
}
}

void build_x(int node_x, int start_x, int end_x) {
    if (start_x != end_x) {
        int mid_x = (start_x + end_x) / 2;
        build_x(node_x * 2, start_x, mid_x);
        build_x(node_x * 2 + 1, mid_x + 1, end_x);
    }
    build_y(node_x, start_x, end_x, 1, 0, m - 1);
}

// Query on y-axis for a fixed x-segment
T query_y(int node_x, int node_y, int start_y, int end_y, int
y1, int y2) {
    if (y1 > end_y || y2 < start_y) return identity();
    if (y1 <= start_y && end_y <= y2) return tree[node_x]
[node_y];
    int mid_y = (start_y + end_y) / 2;
    return merge(
        query_y(node_x, node_y * 2, start_y, mid_y, y1, y2),
        query_y(node_x, node_y * 2 + 1, mid_y + 1, end_y, y1,
y2));
}

// Query on x-axis
T query_x(int node_x, int start_x, int end_x, int x1, int x2,
int y1, int y2) {

```

```

    if (x1 > end_x || x2 < start_x) return identity();
    if (x1 <= start_x && end_x <= x2)
        return query_y(node_x, 1, 0, m - 1, y1, y2);

    int mid_x = (start_x + end_x) / 2;
    return merge(
        query_x(node_x * 2, start_x, mid_x, x1, x2, y1, y2),
        query_x(node_x * 2 + 1, mid_x + 1, end_x, x1, x2, y1,
y2));
}

// Update on y-axis for fixed x-segment
void update_y(int node_x, int start_x, int end_x,
    int node_y, int start_y, int end_y,
    int x, int y, T val) {
    if (start_y == end_y) {
        if (start_x == end_x)
            tree[node_x][node_y] = val;
        else
            tree[node_x][node_y] = merge(
                tree[node_x * 2][node_y],
                tree[node_x * 2 + 1][node_y]);
    } else {
        int mid_y = (start_y + end_y) / 2;
        if (y <= mid_y)
            update_y(node_x, start_x, end_x, node_y * 2, start_y,
mid_y, x, y, val);
        else
            update_y(node_x, start_x, end_x, node_y * 2 + 1, mid_y
+ 1, end_y, x, y, val);

        tree[node_x][node_y] = merge(
            tree[node_x][node_y * 2],
            tree[node_x][node_y * 2 + 1]);
    }
}

```

```
// Update on x-axis
void update_x(int node_x, int start_x, int end_x, int x, int
y, T val) {
    if (start_x != end_x) {
        int mid_x = (start_x + end_x) / 2;
        if (x <= mid_x)
            update_x(node_x * 2, start_x, mid_x, x, y, val);
        else
            update_x(node_x * 2 + 1, mid_x + 1, end_x, x, y, val);
    }
    update_y(node_x, start_x, end_x, 1, 0, m - 1, x, y, val);
}

private:
T merge(T a, T b) { return a + b; } // use min/max/gcd as
needed
T identity() { return 0; }
};
```

2.8. range_update_tree.h

```
template <typename T, typename F>
struct RangeUpdateTree {
    int n;
    vector<T> tree;
    T identity;
    F merge;
    RangeUpdateTree(const vector<T>& arr, T id, F _m)
        : n((int)arr.size()), tree(2 * n), identity(id),
merge(_m) {
        for (int i = 0; i < n; i++)
            tree[n + i] = arr[i];
        for (int i = n - 1; i >= 1; i--)
            tree[i] = merge(tree[2 * i], tree[2 * i + 1]);
    }
    void update(int l, int r, T value) {
```

```
assert(l >= 0 && r < n && l <= r);
for (l += n, r += n; l <= r; l >= 1, r >= 1) {
    if (l & 1)
        tree[l] = merge(value, tree[l]), l++;
    if (!(r & 1))
        tree[r] = merge(value, tree[r]), r--;
}
if (l == r)
    tree[l] = merge(value, tree[l]);
}
T query(int v) {
    T res = tree[v += n];
    for (; v > 1; v >= 1)
        res = merge(res, tree[v >> 1]);
    return res;
}
};
// ex: RangeUpdateTree<int, decltype(join)> v(vi (n, 1e9), 1e9,
join); use auto
// func for join
```

3. Graph

3.1. HLD.h

```
struct HLD {
    int n, timer = 0;
    vi top, tin, p, sub;
    HLD(vvi& adj) : n(sz(adj)), top(n), tin(n), p(n, -1), sub(n,
1) {
        vi ord(n + 1);
        for (int i = 0, t = 0, v = ord[i]; i < n; v = ord[++i])
            for (auto& to : adj[v])
                if (to != p[v])
                    p[to] = v, ord[++t] = to;
        for (int i = n - 1, v = ord[i]; i > 0; v = ord[--i])
            sub[p[v]] += sub[v];
```

```

    for (int v = 0; v < n; v++)
        if (sz(adj[v]))
            iter_swap(begin(adj[v]), max_element(all(adj[v]), [&](int a, int b) {
                return make_pair(a != p[v], sub[a]) <
                        make_pair(b != p[v], sub[b]);
            }));
function<void(int)> dfs = [&](int v) {
    tin[v] = timer++;
    for (auto& to : adj[v])
        if (to != p[v]) {
            top[to] = (to == adj[v][0] ? top[v] : to);
            dfs(to);
        }
};
dfs(0);
}
int lca(int u, int v) {
    return process(u, v, [](int, int) {});
}
template <class B>
int process(int a, int b, B op, bool ignore_lca = false) {
    for (int v;; op(tin[v], tin[b]), b = p[v]) {
        if (tin[a] > tin[b])
            swap(a, b);
        if ((v = top[b]) == top[a])
            break;
    }
    if (int l = tin[a] + ignore_lca, r = tin[b]; l <= r)
        op(l, r);
    return a;
}
template <class B>
void subtree(int v, B op, bool ignore_lca = false) {
    if (sub[v] > 1 or !ignore_lca)
        op(tin[v] + ignore_lca, tin[v] + sub[v] - 1);
}

```

```

    }
};

```

3.2. KthAnc.h

// O(log n) LCA with Kth anc

```

struct LCA {
    int n;
    vvi& adjLists;
    int lg;
    vvi up;
    vi depth;
    LCA(vvi& _adjLists, int root = 0) : n(sz(_adjLists)),
    adjLists(_adjLists) {
        lg = 1;
        int pw = 1;
        while (pw <= n)
            pw <= 1, lg++;
        // lg = 20
        up = vvi(n, vi(lg));
        depth.assign(n, -1);
        function<void(int, int)> parentDFS = [&](int from, int
parent) {
            depth[from] = depth[parent] + 1;
            up[from][0] = parent;
            for (auto to : adjLists[from]) {
                if (to == parent)
                    continue;
                parentDFS(to, from);
            }
        };
        parentDFS(root, root);
        for (int j = 1; j < lg; j++) {
            for (int i = 0; i < n; i++) {
                up[i][j] = up[up[i][j - 1]][j - 1];
            }
        }
    }
}

```

```

}
int kthAnc(int v, int k) {
    int ret = v;
    int pw = 0;
    while (k) {
        if (k & 1)
            ret = up[ret][pw];
        k >>= 1;
        pw++;
    }
    return ret;
}
int lca(int u, int v) {
    if (depth[u] > depth[v])
        swap(u, v);
    v = kthAnc(v, depth[v] - depth[u]);
    if (u == v)
        return v;
    while (up[u][0] != up[v][0]) {
        int i = 0;
        for (; i < lg - 1; i++) {
            if (up[u][i + 1] == up[v][i + 1])
                break;
        }
        u = up[u][i], v = up[v][i];
    }
    return up[u][0];
};

int dist(int u, int v) { return depth[u] + depth[v] - 2 *
depth[lca(u, v)]; }
};

```

3.3. LCA.h

```

// O(1) LCA
struct LCA {
    int T = 0;

```

```

    vi st, path, ret;
    vi en, d;
    RMQ<int> rmq;
    LCA(vector<vi>& C)
        : st(sz(C)), en(sz(C)), d(sz(C)), rmq((dfs(C, 0, -1),
ret)) {}
    void dfs(vvi& adj, int v, int par) {
        st[v] = T++;
        for (auto to : adj[v])
            if (to != par) {
                path.pb(v), ret.pb(st[v]);
                d[to] = d[v] + 1;
                dfs(adj, to, v);
            }
        en[v] = T - 1;
    }
    bool anc(int p, int c) { return st[p] <= st[c] and en[p] >=
en[c]; }
    int lca(int a, int b) {
        if (a == b)
            return a;
        tie(a, b) = minmax(st[a], st[b]);
        return path[rmq.query(a, b - 1)];
    }
    int dist(int a, int b) { return d[a] + d[b] - 2 * d[lca(a,
b)]; }
};

```

3.4. bellman.h

```

struct Edge {
    int a, b, cost;
};

int n, m, v;
vector<Edge> edges;
const int INF = 1000000000;

```

```

void solve() {
    vector<int> d(n, INF);
    d[v] = 0;
    vector<int> p(n, -1);
    int x;
    for (int i = 0; i < n; ++i) {
        x = -1;
        for (Edge e : edges)
            if (d[e.a] < INF)
                if (d[e.b] > d[e.a] + e.cost) {
                    d[e.b] = max(-INF, d[e.a] + e.cost);
                    p[e.b] = e.a;
                    x = e.b;
                }
    }

    if (x == -1)
        cout << "No negative cycle from " << v;
    else {
        int y = x;
        for (int i = 0; i < n; ++i)
            y = p[y];

        vector<int> path;
        for (int cur = y;; cur = p[cur]) {
            path.push_back(cur);
            if (cur == y && path.size() > 1)
                break;
        }
        reverse(path.begin(), path.end());

        cout << "Negative cycle: ";
        for (int u : path)
            cout << u << ' ';
    }
}

```

```

    }
}

```

3.5. bridges.h

```

int n; // number of nodes
vector<vector<int>> adj; // adjacency list of graph

vector<bool> visited;
vector<int> tin, low;
int timer;

void dfs(int v, int p = -1) {
    visited[v] = true;
    tin[v] = low[v] = timer++;
    for (int to : adj[v]) {
        if (to == p)
            continue;
        if (visited[to]) {
            low[v] = min(low[v], tin[to]);
        } else {
            dfs(to, v);
            low[v] = min(low[v], low[to]);
            if (low[to] > tin[v])
                IS_BRIDGE(v, to);
        }
    }
}

void find_bridges() {
    timer = 0;
    visited.assign(n, false);
    tin.assign(n, -1);
    low.assign(n, -1);
    for (int i = 0; i < n; ++i) {
        if (!visited[i])
            dfs(i);
    }
}

```

```

    }
}

// ARTICULATION POINTS:
int n;
vector<vector<int>> adj;
vector<bool> visited;
vector<int> tin, low;
int timer;
void dfs(int v, int p = -1) {
    visited[v] = true;
    tin[v] = low[v] = timer++;
    int children = 0;
    for (int to : adj[v]) {
        if (to == p)
            continue;
        if (visited[to]) {
            low[v] = min(low[v], tin[to]);
        } else {
            dfs(to, v);
            low[v] = min(low[v], low[to]);
            if (low[to] >= tin[v] && p != -1)
                IS_CUTPOINT(v);
            ++children;
        }
    }
    if (p == -1 && children > 1)
        IS_CUTPOINT(v);
}
void find_cutpoints() {
    timer = 0;
    visited.assign(n, false);
    tin.assign(n, -1);
    low.assign(n, -1);
    for (int i = 0; i < n; ++i) {
        if (!visited[i])

```

```

        dfs(i);
    }
}

// arya bridges
void findBridges_dfs(int u, int p, int& time,
vector<vector<int>>& adj,
vector<int>& disc, vector<int>& low,
vector<pair<int, int>>& bridges) {
    disc[u] = low[u] = time++;

    for (int v : adj[u]) {
        if (v == p)
            continue;

        if (disc[v] != -1) {
            low[u] = min(low[u], disc[v]);
        } else {
            findBridges_dfs(v, u, time, adj, disc, low, bridges);
            low[u] = min(low[u], low[v]);
            if (low[v] > disc[u]) {
                bridges.push_back({u, v});
            }
        }
    }
}

vector<pair<int, int>> findBridges(int n, vector<vector<int>>&
adj) {
    vector<int> disc(n, -1), low(n, -1);
    vector<pair<int, int>> bridges;
    int time = 0;

    for (int i = 0; i < n; ++i) {
        if (disc[i] == -1) {
            findBridges_dfs(i, -1, time, adj, disc, low, bridges);

```

```

    }
}
return bridges;
}

```

3.6. dijkstra.h

```

const int INF = 1000000000;
vector<vector<pair<int, int>>> adj;

void dijkstra(int s, vector<int>& d, vector<int>& p) {
    int n = adj.size();
    d.assign(n, INF);
    p.assign(n, -1);

    d[s] = 0;
    set<pair<int, int>> q;
    q.insert({0, s});
    while (!q.empty()) {
        int v = q.begin()->second;
        q.erase(q.begin());

        for (auto edge : adj[v]) {
            int to = edge.first;
            int len = edge.second;

            if (d[v] + len < d[to]) {
                q.erase({d[to], to});
                d[to] = d[v] + len;
                p[to] = v;
                q.insert({d[to], to});
            }
        }
    }
}

```

3.7. dsu.h

```

struct DSU {
    vector<int> p, sz;
    DSU(int n) {
        p.resize(n);
        sz.assign(n, 1);
        for (int i = 0; i < n; ++i) p[i] = i;
    }
    int find(int x) { return p[x] == x ? x : p[x] = find(p[x]); }
    bool unite(int a, int b) {
        a = find(a);
        b = find(b);
        if (a == b) return false;
        if (sz[a] < sz[b]) swap(a, b);
        p[b] = a;
        sz[a] += sz[b];
        return true;
    }
    int size(int x) { return sz[find(x)]; }
};

```

3.8. floyd_washall.h

```

const long long INF = (long long)1e18;
bool floyd_warshall(int n, vector<vector<long long>>& dist) {
    // initialise dist with edge weights, INF if no edge exists
    for (int k = 0; k < n; k++)
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
                if (dist[i][k] < INF && dist[k][j] < INF)
                    dist[i][j] = min(dist[i][j], dist[i][k] + dist[k]
[j]);

    for (int i = 0; i < n; i++)
        if (dist[i][i] < 0)
            return true;
}

```

```
    return false;
}
```

3.9. functional.h

```
#include <cmath>
#include <iostream>
#include <map>
#include <vector>

using std::cout;
using std::endl;
using std::vector;

int main() {
    int planet_num;
    int query_num;
    std::cin >> planet_num >> query_num;
    vector<int> next(planet_num);
    vector<vector<int>> before(planet_num);
    for (int p = 0; p < planet_num; p++) {
        std::cin >> next[p];
        next[p]--;
        before[next[p]].push_back(p);
    }

    /*
     * -2 = We haven't even got to processing this planet yet.
     * -1 = This node is part of a tree.
     * >= 0: the ID of the cycle the planet belongs to.
     */
    vector<int> cycle_id(planet_num, -2);
    // Each map, given a planet #, returns the index of that
    // planet in the
    // cycle.
    vector<std::map<int, int>> cycles;
    for (int p = 0; p < planet_num; p++) {
```

```
        if (cycle_id[p] != -2) {
            continue;
        }
        vector<int> path{p};
        int at = p;
        while (cycle_id[next[at]] == -2) {
            at = next[at];
            cycle_id[at] = -3; // Leave breadcrumbs for this
iteration
            path.push_back(at);
        }

        std::map<int, int> cycle;
        bool in_cycle = false;
        for (int i : path) {
            in_cycle = in_cycle || i == next[at];
            if (in_cycle) {
                cycle[i] = cycle.size();
            }
            cycle_id[i] = in_cycle ? cycles.size() : -1;
        }
        cycles.push_back(cycle);
    }

    /*
     * Precalculate the distance from each planet to its cycle
with BFS.
     * (cyc_dist[p] = 0) if p is part of a cycle.
     */
    vector<int> cyc_dist(planet_num);
    for (int p = 0; p < planet_num; p++) {
        // Check if this planet is part of a cycle.
        if (cycle_id[next[p]] == -1 || cycle_id[p] != -1) {
            continue;
        }
        cyc_dist[p] = 1;
```

```

    vector<int> stack(before[p]);
    while (!stack.empty()) {
        int curr = stack.back();
        stack.pop_back();
        cyc_dist[curr] = cyc_dist[next[curr]] + 1;
        stack.insert(stack.end(), before[curr].begin(),
before[curr].end());
    }
}

// Initialize the binary jumping arrays.
int log2 = std::ceil(std::log2(planet_num));
vector<vector<int>> pow2_ends(planet_num, vector<int>(log2 +
1));
for (int p = 0; p < planet_num; p++) {
    pow2_ends[p][0] = next[p];
}
for (int i = 1; i <= log2; i++) {
    for (int p = 0; p < planet_num; p++) {
        pow2_ends[p][i] = pow2_ends[pow2_ends[p][i - 1]][i - 1];
    }
}

/*
 * Given a starting planet & dist, returns the planet we end
up at
 * if we use the teleporter dist times.
 */
auto advance = [&](int pos, int dist) {
    for (int pow = log2; pow >= 0; pow--) {
        if ((dist & (1 << pow)) != 0) {
            pos = pow2_ends[pos][pow];
        }
    }
    return pos;
};

```

```

for (int q = 0; q < query_num; q++) {
    int u, v; // going from u to v
    std::cin >> u >> v;
    u--;
    v--;
    if (cycle_id[pow2_ends[u][log2]] != cycle_id[pow2_ends[v]
[log2]]) {
        cout << -1 << '\n';
        continue;
    }
    if (cycle_id[u] != -1 || cycle_id[v] != -1) {
        if (cycle_id[v] == -1 && cycle_id[u] != -1) {
            cout << -1 << '\n';
            continue;
        }
        // Handle the 2nd & 3rd cases at the same time.
        int dist = cyc_dist[u];
        int u_cyc = advance(u, cyc_dist[u]); // The root of u's
tree

        std::map<int, int>& cyc = cycles[cycle_id[u_cyc]]; // u
and v's cycle
        int u_ind = cyc[u_cyc];
        int v_ind = cyc[v];
        int diff = u_ind <= v_ind ? v_ind - u_ind : cyc.size() -
(u_ind - v_ind);
        cout << dist + diff << '\n';
    } else {
        if (cyc_dist[v] > cyc_dist[u]) {
            cout << -1 << '\n';
            continue;
        }
        int diff = cyc_dist[u] - cyc_dist[v];
        cout << (advance(u, diff) == v ? diff : -1) << '\n';
    }
}

```

```

    }
}

```

3.10. kosaraju.h

```

vector<int> st;          // - stack holding the unclaimed vertices
vector<int> roots;       // - keeps track of the SCC roots of the
vertices
int timer;              // - dfs timestamp counter
vector<int> t_in;        // - keeps track of the dfs timestamp of
the vertices
vector<int> t_low;       // - keeps track of the lowest t_in of
unclaimed vertices
                        // reachable in the subtree

// implements the tarjan algorithm for strongly connected
components
void dfs(int v, vector<vector<int>> const& adj,
vector<vector<int>>& components) {
    t_low[v] = t_in[v] = timer++;
    st.push_back(v);

    for (auto u : adj[v]) {
        if (t_in[u] == -1) { // tree-edge
            dfs(u, adj, components);
            t_low[v] = min(t_low[v], t_low[u]);
        } else if (roots[u] == -1) { // back-edge, cross-edge or
forward-edge to an unclaimed vertex
            t_low[v] = min(t_low[v], t_in[u]);
        }
    }

    if (t_low[v] == t_in[v]) { // vertex is a root
        components.push_back({v}); // initializes a new component
with root v
        while (true) {
            int u = st.back();

```

```

            st.pop_back();
            roots[u] = v; // claims the vertex
            if (u == v)
                break;
            components.back().push_back(u); // adds vertex u to the
component of v
        }
    }
}

// input: adj -- adjacency list of G
// output: components -- the strongly connected components in G
// output: adj_cond -- adjacency list of G^SCC (by root
vertices)
void strongly_connected_components(vector<vector<int>> const&
adj,
                                vector<vector<int>>&
components,
                                vector<vector<int>>&
adj_cond) {
    components.clear();
    adj_cond.clear();

    int n = adj.size();

    st.clear();
    roots.assign(n, -1);
    timer = 0;
    t_in.assign(n, -1);
    t_low.assign(n, -1);

    // applies the tarjan algorithm to all the vertices
    // adds vertices to the components in reverse topological
order
    for (int v = 0; v < n; v++) {
        if (t_in[v] == -1) {

```

```

        dfs(v, adj, components);
    }
}

// adds edges to the condensation graph
adj_cond.assign(n, {});
for (int v = 0; v < n; v++) {
    for (auto u : adj[v])
        if (roots[v] != roots[u])
            adj_cond[roots[v]].push_back(roots[u]);
}
}

```

3.11. kruskal.h

```

vector<int> parent, rank;

void make_set(int v) {
    parent[v] = v;
    rank[v] = 0;
}

int find_set(int v) {
    if (v == parent[v])
        return v;
    return parent[v] = find_set(parent[v]);
}

void union_sets(int a, int b) {
    a = find_set(a);
    b = find_set(b);
    if (a != b) {
        if (rank[a] < rank[b])
            swap(a, b);
        parent[b] = a;
        if (rank[a] == rank[b])
            rank[a]++;
    }
}

```

```

    }
}

struct Edge {
    int u, v, weight;
    bool operator<(Edge const& other) {
        return weight < other.weight;
    }
};

int n;
vector<Edge> edges;

int cost = 0;
vector<Edge> result;
parent.resize(n);
rank.resize(n);
for (int i = 0; i < n; i++)
    make_set(i);

sort(edges.begin(), edges.end());

for (Edge e : edges) {
    if (find_set(e.u) != find_set(e.v)) {
        cost += e.weight;
        result.push_back(e);
        union_sets(e.u, e.v);
    }
}

```

3.12. planet_queries.h

```

#include <cmath>
#include <iostream>
#include <map>
#include <vector>

```

```

using std::cout;
using std::endl;
using std::vector;

int main() {
    int planet_num;
    int query_num;
    std::cin >> planet_num >> query_num;
    vector<int> next(planet_num);
    vector<vector<int>> before(planet_num);
    for (int p = 0; p < planet_num; p++) {
        std::cin >> next[p];
        next[p]--;
        before[next[p]].push_back(p);
    }

    /*
     * -2 = We haven't even got to processing this planet yet.
     * -1 = This node is part of a tree.
     * >= 0: the ID of the cycle the planet belongs to.
     */
    vector<int> cycle_id(planet_num, -2);
    // Each map, given a planet #, returns the index of that
    planet in the
    // cycle.
    vector<std::map<int, int>> cycles;
    for (int p = 0; p < planet_num; p++) {
        if (cycle_id[p] != -2) {
            continue;
        }
        vector<int> path{p};
        int at = p;
        while (cycle_id[next[at]] == -2) {
            at = next[at];
            cycle_id[at] = -3; // Leave breadcrumbs for this
iteration

```

```

        path.push_back(at);
    }

    std::map<int, int> cycle;
    bool in_cycle = false;
    for (int i : path) {
        in_cycle = in_cycle || i == next[at];
        if (in_cycle) {
            cycle[i] = cycle.size();
        }
        cycle_id[i] = in_cycle ? cycles.size() : -1;
    }
    cycles.push_back(cycle);
}

/*
 * Precalculate the distance from each planet to its cycle
with BFS.
 * (cyc_dist[p] = 0) if p is part of a cycle.
 */
vector<int> cyc_dist(planet_num);
for (int p = 0; p < planet_num; p++) {
    // Check if this planet is part of a cycle.
    if (cycle_id[next[p]] == -1 || cycle_id[p] != -1) {
        continue;
    }
    cyc_dist[p] = 1;
    vector<int> stack(before[p]);
    while (!stack.empty()) {
        int curr = stack.back();
        stack.pop_back();
        cyc_dist[curr] = cyc_dist[next[curr]] + 1;
        stack.insert(stack.end(), before[curr].begin(),
before[curr].end());
    }
}

```

```

// Initialize the binary jumping arrays.
int log2 = std::ceil(std::log2(planet_num));
vector<vector<int>> pow2_ends(planet_num, vector<int>(log2 +
1));
for (int p = 0; p < planet_num; p++) {
    pow2_ends[p][0] = next[p];
}
for (int i = 1; i <= log2; i++) {
    for (int p = 0; p < planet_num; p++) {
        pow2_ends[p][i] = pow2_ends[pow2_ends[p][i - 1]][i - 1];
    }
}

/*
 * Given a starting planet & dist, returns the planet we end
up at
 * if we use the teleporter dist times.
 */
auto advance = [&](int pos, int dist) {
    for (int pow = log2; pow >= 0; pow--) {
        if ((dist & (1 << pow)) != 0) {
            pos = pow2_ends[pos][pow];
        }
    }
    return pos;
};

for (int q = 0; q < query_num; q++) {
    int u, v; // going from u to v
    std::cin >> u >> v;
    u--;
    v--;
    if (cycle_id[pow2_ends[u][log2]] != cycle_id[pow2_ends[v]
[log2]]) {
        cout << -1 << '\n';
        continue;
    }
}

```

```

}
if (cycle_id[u] != -1 || cycle_id[v] != -1) {
    if (cycle_id[v] == -1 && cycle_id[u] != -1) {
        cout << -1 << '\n';
        continue;
    }
    // Handle the 2nd & 3rd cases at the same time.
    int dist = cyc_dist[u];
    int u_cyc = advance(u, cyc_dist[u]); // The root of u's
tree

    std::map<int, int>& cyc = cycles[cycle_id[u_cyc]]; // u
and v's cycle
    int u_ind = cyc[u_cyc];
    int v_ind = cyc[v];
    int diff = u_ind <= v_ind ? v_ind - u_ind : cyc.size() -
(u_ind - v_ind);
    cout << dist + diff << '\n';
} else {
    if (cyc_dist[v] > cyc_dist[u]) {
        cout << -1 << '\n';
        continue;
    }
    int diff = cyc_dist[u] - cyc_dist[v];
    cout << (advance(u, diff) == v ? diff : -1) << '\n';
}
}
}

```

4. Geometry

4.1. circle.h

```

// Circle Geometry: Tangents and Circle Intersections
struct Circle {
    Pd center;
    double radius;
}

```

```

    Circle(Pd center, double radius) : center(center),
    radius(radius) {}
};

bool pointInCircle(const Circle& circle, Pd p) {
    return (p - circle.center).dist2() <=
        circle.radius * circle.radius + GEOMETRY_EPS;
}

bool pointOnCircle(const Circle& circle, Pd p) {
    return abs((p - circle.center).dist() - circle.radius) <=
    GEOMETRY_EPS;
}

// Length of either tangent from p to the circle.
// Returns false when p is strictly inside the circle.
bool tangentLength(const Circle& circle, Pd p, double& length)
{
    double d2 = (p - circle.center).dist2();
    double r2 = circle.radius * circle.radius;
    if (d2 < r2 - GEOMETRY_EPS) return false;
    length = sqrt(max(0.0, d2 - r2));
    return true;
}

// Tangency points from p to the circle.
// On the circle, both output points are set to p.
// Returns false when p is strictly inside the circle.
bool tangentPoints(const Circle& circle, Pd p, Pd& t1, Pd& t2)
{
    Pd v = p - circle.center;
    double d2 = v.dist2();
    double r2 = circle.radius * circle.radius;
    if (d2 < r2 - GEOMETRY_EPS) return false;
    if (d2 < GEOMETRY_EPS) return false;

```

```

    double along = r2 / d2;
    double height = circle.radius * sqrt(max(0.0, d2 - r2)) / d2;
    Pd perpendicular(-v.y, v.x);
    t1 = circle.center + v * along + perpendicular * height;
    t2 = circle.center + v * along - perpendicular * height;
    return true;
}

// Intersections of an infinite line and a circle.
// The output has zero, one, or two points and returns whether
any exist.
bool circleLineIntersections(const Circle& circle, Pd a, Pd b,
    vector<Pd>& result) {
    result.clear();
    Pd foot = projectionOnLine(a, b, circle.center);
    double distance = (foot - circle.center).dist();
    if (distance > circle.radius + GEOMETRY_EPS) return false;

    Pd direction = b - a;
    double length = direction.dist();
    if (length < GEOMETRY_EPS) return pointOnCircle(circle, a) &&
        (result.push_back(a),
true);
    direction = direction / length;
    double offset = sqrt(max(0.0, circle.radius * circle.radius -
        distance * distance));
    result.push_back(foot - direction * offset);
    if (offset > GEOMETRY_EPS) result.push_back(foot + direction
* offset);
    return true;
}

// Intersections of two circles. Coincident circles have
infinitely many
// intersections and return false because there is no finite

```

result list.

```
bool circleCircleIntersections(const Circle& first, const
Circle& second,
                                vector<Pd>& result) {
    result.clear();
    Pd delta = second.center - first.center;
    double distance = delta.dist();
    double r1 = first.radius, r2 = second.radius;

    if (distance < GEOMETRY_EPS) return false;
    if (distance > r1 + r2 + GEOMETRY_EPS) return false;
    if (distance < abs(r1 - r2) - GEOMETRY_EPS) return false;

    double along = (r1 * r1 - r2 * r2 + distance * distance) /
        (2.0 * distance);
    double height2 = r1 * r1 - along * along;
    if (height2 < -GEOMETRY_EPS) return false;

    Pd base = first.center + delta * (along / distance);
    if (height2 <= GEOMETRY_EPS) {
        result.push_back(base);
        return true;
    }

    double height = sqrt(height2);
    Pd perpendicular(-delta.y / distance, delta.x / distance);
    result.push_back(base + perpendicular * height);
    result.push_back(base - perpendicular * height);
    return true;
}
```

4.2. convex_hull.h

```
// Convex Hull (Monotone Chain)
template <class T>
vector<Point<T>> convexHull(vector<Point<T>> pts) {
    int n = pts.size(), k = 0;
```

```
    if (n <= 2) return pts;
    vector<Point<T>> hull(2 * n);
    sort(pts.begin(), pts.end(), [](const Point<T>& a, const
Point<T>& b) {
        return a.x < b.x || (a.x == b.x && a.y < b.y);
    });
    for (int i = 0; i < n; i++) {
        while (k >= 2 && crossProduct(hull[k - 2], hull[k - 1],
pts[i]) <= 0) k--;
        hull[k++] = pts[i];
    }
    for (int i = n - 2, t = k + 1; i >= 0; i--) {
        while (k >= t && crossProduct(hull[k - 2], hull[k - 1],
pts[i]) <= 0) k--;
        hull[k++] = pts[i];
    }
    hull.resize(k - 1);
    return hull;
}
```

4.3. intersections.h

```
// Line and Segment Intersections
template <class T>
T crossProduct(Point<T> a, Point<T> b, Point<T> c) {
    return (b - a).cross(c - a);
}

// 1 if left, -1 if right, 0 if collinear
template <class T>
int orientation(Point<T> a, Point<T> b, Point<T> c) {
    T cp = crossProduct(a, b, c);
    return (cp > 0) - (cp < 0);
}

// Check if point p is on segment ab
template <class T>
```

```

bool onSegment(Point<T> a, Point<T> b, Point<T> p) {
    return orientation(a, b, p) == 0 && p.x >= min(a.x, b.x) &&
p.x <= max(a.x, b.x) && p.y >= min(a.y, b.y) && p.y <= max(a.y,
b.y);
}

// Check if segment ab and cd intersect
template <class T>
bool segmentIntersect(Point<T> a, Point<T> b, Point<T> c,
Point<T> d) {
    int o1 = orientation(a, b, c), o2 = orientation(a, b, d);
    int o3 = orientation(c, d, a), o4 = orientation(c, d, b);
    if (o1 != o2 && o3 != o4) return true;
    if (onSegment(a, b, c) || onSegment(a, b, d) || onSegment(c,
d, a) || onSegment(c, d, b)) return true;
    return false;
}

constexpr double GEOMETRY_EPS = 1e-9;

// Projection of p onto the infinite line through a and b.
Pd projectionOnLine(Pd a, Pd b, Pd p) {
    Pd ab = b - a;
    double ab2 = ab.dist2();
    if (ab2 < GEOMETRY_EPS) return a;
    return a + ab * ((p - a).dot(ab) / ab2);
}

// Closest point to p on the closed segment [a, b].
Pd closestPointOnSegment(Pd a, Pd b, Pd p) {
    Pd ab = b - a;
    double ab2 = ab.dist2();
    if (ab2 < GEOMETRY_EPS) return a;
    double t = (p - a).dot(ab) / ab2;
    t = max(0.0, min(1.0, t));
    return a + ab * t;
}

```

```

}

double distancePointToLine(Pd a, Pd b, Pd p) {
    Pd ab = b - a;
    double length = ab.dist();
    if (length < GEOMETRY_EPS) return (p - a).dist();
    return abs(ab.cross(p - a)) / length;
}

double distancePointToSegment(Pd a, Pd b, Pd p) {
    return (p - closestPointOnSegment(a, b, p)).dist();
}

// Infinite line intersection. Returns false for parallel or
coincident lines.
bool lineIntersection(Pd a, Pd b, Pd c, Pd d, Pd& res) {
    double cp = (b - a).cross(d - c);
    if (abs(cp) < GEOMETRY_EPS) return false;
    double t = (c - a).cross(d - c) / cp;
    res = a + (b - a) * t;
    return true;
}

// Returns the unique intersection point of closed segments [a,
b] and [c, d].
// Collinear overlap has no unique point and therefore returns
false.
bool segmentIntersectionPoint(Pd a, Pd b, Pd c, Pd d, Pd& res)
{
    if (!lineIntersection(a, b, c, d, res)) return false;
    auto inRange = [](double value, double left, double right) {
        return value >= min(left, right) - GEOMETRY_EPS &&
            value <= max(left, right) + GEOMETRY_EPS;
    };
    return inRange(res.x, a.x, b.x) && inRange(res.y, a.y, b.y)
&&

```

```

        inRange(res.x, c.x, d.x) && inRange(res.y, c.y, d.y);
    }

    // Length of the overlap of two collinear segments. Returns
    // zero otherwise.
    double collinearSegmentIntersectionLength(Pd a, Pd b, Pd c, Pd
    d) {
        Pd ab = b - a;
        double ab2 = ab.dist2();
        if (ab2 < GEOMETRY_EPS) return 0.0;
        if (abs(ab.cross(c - a)) > GEOMETRY_EPS ||
            abs(ab.cross(d - a)) > GEOMETRY_EPS)
            return 0.0;

        double tc = (c - a).dot(ab) / ab2;
        double td = (d - a).dot(ab) / ab2;
        double left = max(0.0, min(tc, td));
        double right = min(1.0, max(tc, td));
        return max(0.0, right - left) * sqrt(ab2);
    }

```

4.4. point.h

```

// Geometric Primitives: Point, Cross Product, Dot Product,
// Rotation
template <class T>
struct Point {
    T x, y;
    Point() : x(0), y(0) {}
    Point(T x, T y) : x(x), y(y) {}
    Point operator+(const Point& p) const { return Point(x + p.x,
    y + p.y); }
    Point operator-(const Point& p) const { return Point(x - p.x,
    y - p.y); }
    Point operator*(T c) const { return Point(x * c, y * c); }
    Point operator/(T c) const { return Point(x / c, y / c); }

```

```

    T dot(const Point& p) const { return x * p.x + y * p.y; }
    T cross(const Point& p) const { return x * p.y - y * p.x; }
    T dist2() const { return x * x + y * y; }
    double dist() const { return sqrt(dist2()); }

    // Rotate point by angle rad (counter-clockwise)
    Point<double> rotate(double angle) const {
        return Point<double>(
            x * cos(angle) - y * sin(angle),
            x * sin(angle) + y * cos(angle));
    }
};

```

```

using P = Point<long long>;
using Pd = Point<double>;

```

4.5. polygon.h

```

// Polygon Math: Shoelace, Point in Polygon
template <class T>
double polygonArea(const vector<Point<T>>& p) {
    T res = 0;
    for (int i = 0, n = p.size(); i < n; i++) {
        res += p[i].cross(p[(i + 1) % n]);
    }
    return abs(res) / 2.0;
}

// Ray casting algorithm for Point in Polygon
// Returns true if strictly inside, handles boundary separately
template <class T>
bool pointInPolygon(const vector<Point<T>>& poly, Point<T> p) {
    bool in = false;
    int n = poly.size();
    for (int i = 0, j = n - 1; i < n; j = i++) {
        if (onSegment(poly[i], poly[j], p)) return true; //
        // Boundary check (optional)
    }
}

```

```

        if (((poly[i].y > p.y) != (poly[j].y > p.y)) &&
            (p.x < (poly[j].x - poly[i].x) * (p.y - poly[i].y) /
(double)(poly[j].y - poly[i].y) + poly[i].x))
            in = !in;
    }
    return in;
}

```

5. Number Theory

5.1. ModularArithmetic.h

```
const long long MOD = 1e9 + 7;
```

```

long long mod_mul(long long a, long long b, long long mod =
MOD) {
    a = a % mod;
    b = b % mod;
    return ((a * b) % mod) + mod) % mod;
}

```

```

long long mod_add(long long a, long long b, long long mod =
MOD) {
    a = a % mod;
    b = b % mod;
    return ((a + b) % mod) + mod) % mod;
}

```

```

long long mod_sub(long long a, long long b, long long mod =
MOD) {
    a = a % mod;
    b = b % mod;
    return ((a - b) % mod) + mod) % mod;
}

```

```

int modexp(int base, int exp, int mod = MOD) {
    int res = 1;

```

```

    base %= mod;
    while (exp > 0) {
        if (exp & 1) res = mod_mul(res, base, mod);
        base = mod_mul(base, base, mod);
        exp >>= 1;
    }
    return res;
}

```

```

int inv(int a, int m = MOD) {
    return modexp(a, m - 2, m);
}

```

```

long long mod_div(long long a, long long b, long long mod =
MOD) {
    return mod_mul(a, inv(b, mod), mod);
}

```

```

vi fact(int n, int mod = MOD) {
    vi f(n + 1, 1);
    for (int i = 2; i <= n; i++)
        f[i] = mod_mul(f[i - 1], i, mod);
    return f;
}

```

```

vi invfact(int n, int mod = MOD) {
    vi facts = fact(n, mod);
    vi invf(n + 1, 1);
    invf[n] = modexp(facts[n], mod - 2, mod);
    for (int i = n - 1; i >= 0; i--)
        invf[i] = mod_mul(invf[i + 1], i + 1, mod);
    return invf;
}

```

```
vector<int> comb_fact, comb_invfact;
```

```

void init_comb(int n, int mod = MOD) {
    comb_fact = fact(n, mod);
    comb_invfact = invfact(n, mod);
}

int nCr(int n, int r, int mod = MOD) {
    if (r < 0 || r > n) return 0;
    int res = mod_mul(comb_fact[n], comb_invfact[r], mod);
    return mod_mul(res, comb_invfact[n - r], mod);
}

```

5.2. bit_bns.h

// --- Bit Binary Search in $O(\log(n))$ ---

```
const int M = 20 const int N = 1 << M
```

```

                                int
                                lower_bound(int val) {

    int ans = 0, sum = 0;
    for (int i = M - 1; i >= 0; i--) {
        int x = ans + (1 << i);
        if (sum + bit[x] < val)
            ans = x, sum += bit[x];
    }

    return ans + 1;
}

```

5.3. knapsack.h

```

for (int i = 1; i <= n; i++)
    for (int j = W; j >= w[i]; j--)
        f[j] = max(f[j], f[j - w[i]] + v[i]);

```

5.4. matrix_expo.h

```

int** matrixmul(int** matrix1, int** matrix2) {
    int** matrix3 = new int*[2];
    for (int i = 0; i < 2; i++)

```

```

        matrix3[i] = new int[2];

        matrix3[0][0] =
            (matrix1[0][0] * matrix2[0][0]) + (matrix1[0][1] *
matrix2[1][0]);
        matrix3[0][1] =
            (matrix1[0][0] * matrix2[0][1]) + (matrix1[0][1] *
matrix2[1][1]);
        matrix3[1][0] =
            (matrix1[1][0] * matrix2[0][0]) + (matrix1[1][1] *
matrix2[1][0]);
        matrix3[1][1] =
            (matrix1[1][0] * matrix2[0][1]) + (matrix1[1][1] *
matrix2[1][1]);
        matrix3[0][0] %= M;
        matrix3[1][0] %= M;
        matrix3[0][1] %= M;
        matrix3[1][1] %= M;

    return matrix3;
}

```

```

int** matrixexpo(int** matrix, int n, int** ans) {
    while (n > 0) {
        if (n % 2 == 1)
            ans = matrixmul(ans, matrix);
        matrix = matrixmul(matrix, matrix);
        n /= 2;
    }
    return ans;
}

```

5.5. my_math.h

```

using u64 = uint64_t;
using i64 = int64_t;

```

```

u64 mult(u64 a, u64 b, u64 m = MOD) {
    // performs modular multiplication (a*b)%m avoiding overflow
    i64 ret = a * b - m * (u64)(1.L / m * a * b);
    return ret + m * (ret < 0) - m * (ret >= (i64)m);
}

u64 pw(u64 b, u64 e, u64 m = MOD) {
    // performs modular exponentiation (b^e)%m avoiding overflow
    using the above
    // mult function u can use ur bexp fnc as well
    u64 ret = 1;
    for (; e; b = mult(b, b, m), e >>= 1)
        if (e & 1)
            ret = mult(ret, b, m);
    return ret;
}

bool isPrime(u64 n) {
    // miller rabin primality test for 64 bit integers
    if (n < 2 || n % 6 % 4 != 1)
        return (n | 1) == 3;
    u64 A[] = {2, 325, 9375, 28178, 450775, 9780504, 1795265022},
        s = __builtin_ctzll(n - 1), d = n >> s;
    for (u64 a : A) {
        // note we are reducing a mod n as a might be larger than n
        // so it is correct
        // as per the condn of less than n-1
        u64 p = pw(a % n, d, n), i = s;
        while (p != 1 && p != n - 1 && a % n && i--)
            p = mult(p, p, n);
        if (p != n - 1 && i != s)
            return 0;
    }
    return 1;
}

```

```

using ull = u64;
ull pollard(ull n) {
    // pollard's rho algo which is probabilistic but works well
    in practice this
    // will return a nontrivial factor of n
    ull x = 0, y = 0, t = 30, prd = 2, i = 1, q;
    auto f = [&](ull x) { return mult(x, x, n) + i; };
    while (t++ % 40 || __gcd(prd, n) == 1) {
        if (x == y)
            x = ++i, y = f(x);
        if ((q = mult(prd, max(x, y) - min(x, y), n)))
            prd = q;
        x = f(x), y = f(f(y));
    }
    return __gcd(prd, n);
}

vector<ull> factor(ull n) {
    // returns the prime factorization of n in O(n^0.25) time
    if (n == 1)
        return {};
    if (isPrime(n))
        return {n};
    ull x = pollard(n);
    auto l = factor(x), r = factor(n / x);
    l.insert(l.end(), all(r));
    return l;
}

```

5.6. some_dp.h

```

// LIS
int lis(vector<int> const& a) {
    int n = a.size();
    const int INF = 1e9;
    vector<int> d(n + 1, INF);
    d[0] = -INF;
}

```

```

for (int i = 0; i < n; i++) {
    int l = upper_bound(d.begin(), d.end(), a[i]) - d.begin();
    if (d[l - 1] < a[i] && a[i] < d[l])
        d[l] = a[i];
}

int ans = 0;
for (int l = 0; l <= n; l++) {
    if (d[l] < INF)
        ans = l;
}
return ans;
}
// or segtree lol

```

5.7. spf.h

```

int MX = 1e7 + 1;
vi spf(MX + 1, INT32_MAX);
vector<int> is_prime(MX + 1, 1);
void sieve(int n = MX) {
    is_prime[0] = is_prime[1] = 0;
    int cnt = 1;
    for (int i = 2; i <= n; i++) {
        if (is_prime[i]) {
            for (int j = i * i; j <= n; j += i) {
                is_prime[j] = 0;
                spf[j] = min(i, spf[j]);
            }
            is_prime[i] = cnt;
            cnt++;
        }
    }
    return;
}

```

6. Strings

6.1. Trie.h

```

class TrieNode {
public:
    unordered_map<char, TrieNode*> children;
    bool isEndOfWord;

    TrieNode() : isEndOfWord(false) {}
};

class Trie {
private:
    TrieNode* root;

public:
    Trie() { root = new TrieNode(); }
    void insert(const string& word) {
        TrieNode* node = root;
        for (char ch : word) {
            if (node->children.find(ch) == node->children.end()) {
                node->children[ch] = new TrieNode();
            }
            node = node->children[ch];
        }
        node->isEndOfWord = true;
    }
    bool search(const string& word) {
        TrieNode* node = root;
        for (char ch : word) {
            if (node->children.find(ch) == node->children.end()) {
                return false;
            }
            node = node->children[ch];
        }
        return node->isEndOfWord;
    }
}

```

```

bool startsWith(const string& prefix) {
    TrieNode* node = root;
    for (char ch : prefix) {
        if (node->children.find(ch) == node->children.end()) {
            return false;
        }
        node = node->children[ch];
    }
    return true;
};

```

6.2. hash.h

```

template <int MOD, int P>
struct RH {
    // using H1 = RH<1000000007, 91138233>;
    // using H2 = RH<1000000009, 97266353>;
    vector<long long> h, p;
    RH(const string& s) {
        int n = s.size();
        h.resize(n + 1, 0);
        p.resize(n + 1, 0);
        p[0] = 1;
        for (int i = 0; i < n; i++) {
            h[i + 1] = (h[i] * P + s[i]) % MOD;
            p[i + 1] = p[i] * P % MOD;
        }
    }
    long long get(int l, int r) { // [l, r]
        long long res = (h[r + 1] - h[l] * p[r - l + 1]) % MOD;
        return res < 0 ? res + MOD : res;
    }
};

```

6.3. kmp.h

```

vector<int> prefix_function(string s) {
    int n = (int)s.length();
    vector<int> pi(n);
    for (int i = 1; i < n; i++) {
        int j = pi[i - 1];
        while (j > 0 && s[i] != s[j])
            j = pi[j - 1];
        if (s[i] == s[j])
            j++;
        pi[i] = j;
    }
    return pi;
}

vector<int> KMP(string text, string pattern) {
    string s = pattern + "#" + text;
    vector<int> pi = prefix_function(s);
    vector<int> matches;
    int p = pattern.length();

    for (int i = 0; i < s.length(); i++) {
        if (pi[i] == p) {
            int match_pos = i - 2 * p;
            matches.push_back(match_pos); // 0-based index in text
        }
    }
    return matches;
}

```

7. Theory

7.1. Combinatorics

Binomial Theorem:

$$(x + y)^n = \sum_{k=0}^n \binom{n}{k} x^{n-k} y^k$$

Expands powers of a binomial.

Combinations (nCr):

$$\binom{n}{r} = \frac{n!}{r!(n-r)!}$$

Number of ways to choose r items from n distinct items without replacement, order does not matter.

Permutations (nPr):

$$P(n, r) = \frac{n!}{(n-r)!}$$

Number of ways to arrange r items from n distinct items, order matters.

Stars and Bars: Used to find the number of integer solutions to $x_1 + x_2 + \dots + x_n = k$.

- Case $x_i \geq 0$ (non-negative integers):

$$\binom{n+k-1}{n-1}$$

or

$$\binom{n+k-1}{k}$$

- Case $x_i \geq 1$ (positive integers):

$$\binom{k-1}{n-1}$$

Catalan Numbers (C_n):

$$C_n = \frac{1}{n+1} \binom{2n}{n} = \binom{2n}{n} - \binom{2n}{n+1}$$

- Number of valid parenthesis sequences of length $2n$.
- Number of structurally unique binary trees with n nodes is C_n .
- Number of rooted ordered trees with n nodes is $C_{\{n-1\}}$.

Derangements: Permutations where no element appears in its original position.

- Recurrence:

$$D_n = (n-1)(D_{\{n-1\}} + D_{\{n-2\}})$$

with $D_1 = 0, D_2 = 1$.

- General Formula:

$$D_n = n! \sum_{i=0}^n \frac{(-1)^i}{i!}$$

Principle of Inclusion-Exclusion (PIE): Used to count the number of elements in the union of overlapping sets.

$$|A \cup B \cup C| = |A| + |B| + |C| - |A \cap B| - |A \cap C| - |B \cap C| + |A \cap B \cap C|$$

7.2. Geometry

Pick's Theorem: Finds the area of a polygon whose vertices are all on integer grid points.

$$\text{Area} = I + \frac{B}{2} - 1$$

Where I is the number of interior grid points and B is the number of boundary grid points.

7.3. Geometry

Points and vectors: Treat a point (x, y) as a vector. For points a, b , and c , the vector from a to b is $b - a$.

Dot product:

$$u \cdot v = u_x v_x + u_y v_y$$

The dot product measures how much one vector points in the direction of another. It is useful for projections and distances along a line.

Cross product in 2D:

$$u \times v = u_x v_y - u_y v_x$$

For a directed line from a to b , compute the 2D cross product $\text{cross}(b - a, c - a)$:

- Positive means c is to the left of $a \rightarrow b$.
- Negative means c is to the right.
- Zero means the three points are collinear.

The magnitude is twice the area of triangle abc . In code this is $(b - a) \cdot \text{cross}(c - a)$, or the helper `crossProduct(a, b, c)`.

Orientation and segment tests: `orientation(a, b, c)` returns 1, -1, or 0 for left, right, or collinear. `onSegment(a, b, p)` additionally checks the bounding box, so it identifies a point on the closed segment rather than anywhere on its infinite line. Two closed segments intersect when their endpoints lie on opposite sides of each other, including the collinear endpoint cases handled by `onSegment`.

Projection and closest point: For a point p and line through a, b , the projection parameter is

$$t = \frac{(p - a) \cdot (b - a)}{(b - a) \cdot (b - a)}$$

and the projected point is $a + t(b - a)$.

`projectionOnLine` allows any real t . For a segment, clamp t to $[0, 1]$; `closestPointOnSegment` does this. Therefore:

- `distancePointToLine` uses perpendicular distance

$$|(b_x - a_x)(p_y - a_y) - (b_y - a_y)(p_x - a_x)| / |b - a|$$

- `distancePointToSegment` measures the distance to the closest projected point, or to an endpoint when the projection lies outside the segment.

Line intersection: Write the lines as $a + t(b - a)$ and $c + s(d - c)$. Let den be the cross product of $b - a$ and $d - c$. If den is zero, the lines are parallel or coincident. Otherwise, the parameter t is the cross product of $c - a$ and $d - c$, divided by den , and the intersection is $a + t(b - a)$. `lineIntersection` returns this point for infinite lines, while `segmentIntersectionPoint` also checks that the point lies inside both segments. Collinear overlapping segments do not have one unique intersection point; `collinearSegmentIntersectionLength` returns the length of their overlap instead.

Circle geometry: A circle is represented by a center o and radius r . A point p is inside or on the circle when $|p - o| \leq r$.

For an external point p , the tangent triangle is right-angled at the tangent point. Therefore the length of either tangent is

$$\sqrt{|p - o|^2 - r^2}$$

- `tangentLength` returns this length and fails when the point is strictly inside. `tangentPoints` returns the one tangent point when p is on the circle, or the two tangent points when p is outside.

To intersect a line and a circle, first project the circle center onto the line. If the distance from the center to the line is greater than r , there is no intersection. Otherwise move from the projection in both line directions by

$$\sqrt{r^2 - d^2}$$

, where d is the distance from the center to the line.

`circleLineIntersections` returns zero, one, or two points.

For two circles with center distance d , let

$$x = \frac{r_1^2 - r_2^2 + d^2}{2d}$$

. The common chord is perpendicular to the line of centers, and its half-length is

$$h = \sqrt{r_1^2 - x^2}$$

. `circleCircleIntersections` returns zero, one, or two points.

Coincident circles have infinitely many intersections, so the function returns false for that case.

Convex hull note: The monotone chain hull repeatedly removes the last point while the cross product is non-left-turning. Using `<= 0` removes collinear boundary points; use `< 0` if all collinear boundary points should be retained.